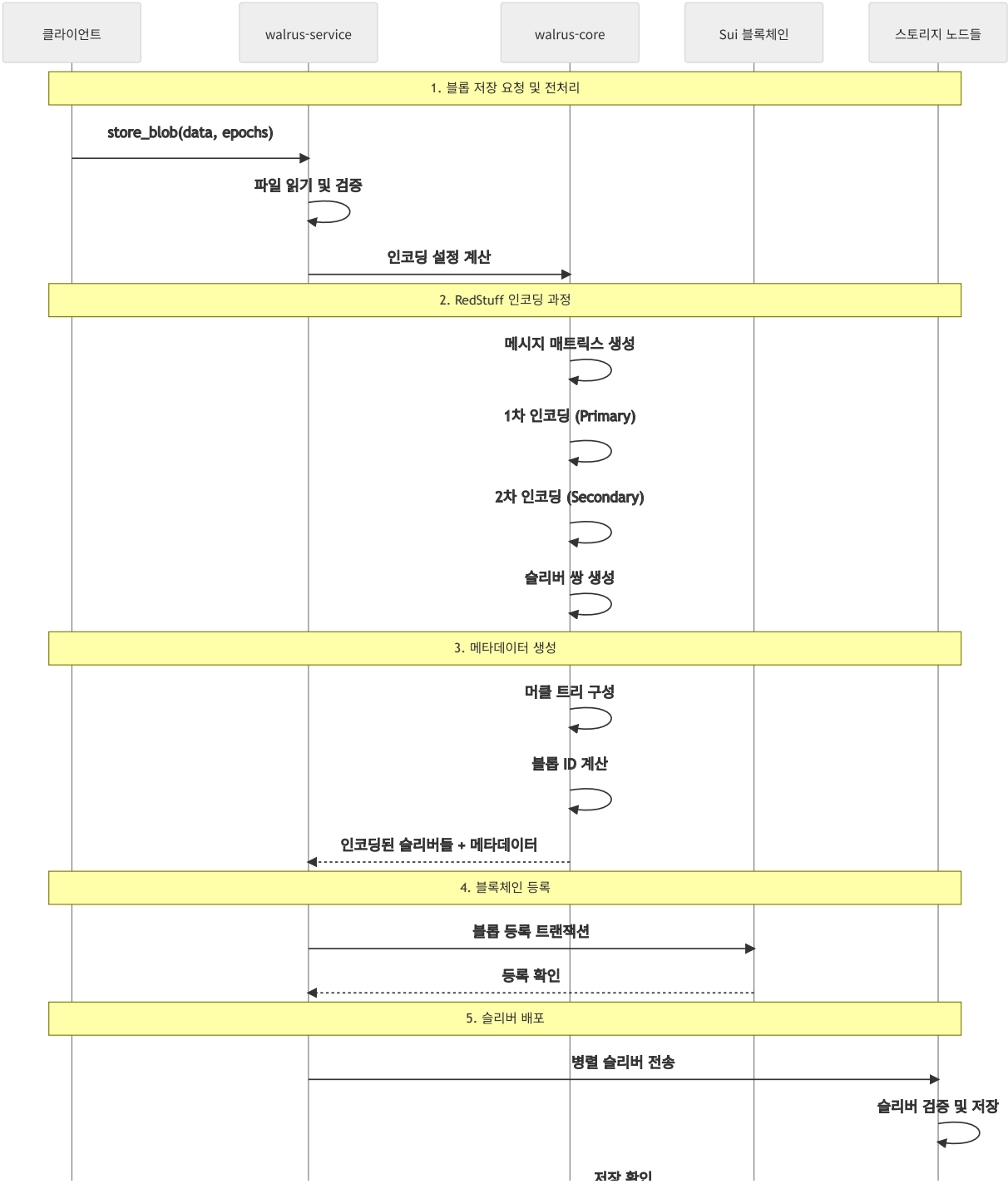
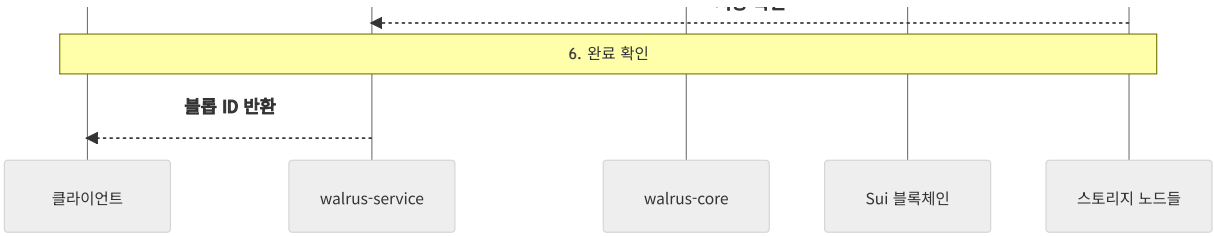
데이터 저장 프로세스 상세 분석

개요

본 문서는 Walrus 시스템에서 클라이언트가 블롭(blob) 저장을 요청한 순간부터 모든 스토리지 노드에 데이터가 안전하게 저장되기까지의 전체 과정을 상세히 분석합니다. 이 과정은 RedStuff 인코딩 알고리즘의 수학적 원리, 슬리버 생성 및 배포, 메타데이터 관리, 그리고 다양한 최적화 기법들을 포함합니다.

전체 데이터 저장 프로세스 개요


클라이언트
walrus-service
walrus-core
Sui 블록체인
스토리지 노드들
1. 블롭 저장 요청 및 전처리
store_blob(data, epochs)
파일 읽기 및 검증
인코딩 설정 계산
2. RedStuff 인코딩 과정
메시지 매트릭스 생성
1차 인코딩 (Primary)
2차 인코딩 (Secondary)
슬리버 쌍 생성
3. 메타데이터 생성
머클 트리 구성
블롭 ID 계산
인코딩된 슬리버들 + 메타데이터
4. 블록체인 등록
블롭 등록 트랜잭션
등록 확인
5. 슬리버 배포
병렬 슬리버 전송
슬리버 검증 및 저장
저장 확인



1. 클라이언트 측 저장 요청 처리

1.1 CLI 명령어 처리

클라이언트가 `walrus store` 명령을 실행하면 다음과 같은 과정이 진행됩니다:

```
// crates/walrus-service/src/client/cli/runner.rs
pub(crate) async fn store(
self,
files: Vec<PathBuf>,
epoch_arg: EpochArg,
dry_run: bool,
store_optimizations: StoreOptimizations,
persistence: BlobPersistence,
```

```
post_store: PostStoreAction,

encoding_type: Option<EncodingType>,

upload_relay: Option<Url>,

confirmation: UserConfirmation,

) -> Result<()> {

// 1. 매개변수 검증

epoch_arg.exactly_one_is_some()?;

if  encoding_type.is_some_and(|encoding| !encoding.is_supported()) {

anyhow::bail!(ClientErrorKind::UnsupportedEncodingType(encoding_type.unwrap()));

}



// 2. 클라이언트 초기화

let  client = get_contract_client(self.config?, self.wallet, self.gas_budget, &N

let  system_object = client.sui_client().read_client.get_system_object().await?;

let  epochs_ahead = get_epochs_ahead(epoch_arg, system_object.max_epochs_ahead()



// 3. 파일 읽기 및 블롭 생성

let  blobs = files

.into_iter()

.map(|file| read_blob_from_file(&file).map(|blob| (file, blob)))

.collect::<Result<Vec<(PathBuf, Vec<u8>)>>>()?;



// 4. 저장 매개변수 설정
```

```rust
let mut store_args = StoreArgs::new(
encoding_type.unwrap_or(DEFAULT_ENCODING),
epochs_ahead,
store_optimizations,
persistence,
post_store,
);

// 5. 실제 저장 실행
let results = client
.reserve_and_store_blobs_retry_committees_with_path(&blobs, &store_args)
.await?;
}
```

1.2 파일 전처리 및 검증

```rust
fn read_blob_from_file(file_path: &PathBuf) -> Result<Vec<u8>> {
// 파일 크기 확인
let metadata = std::fs::metadata(file_path)?;
let file_size = metadata.len();
// 최대 블롭 크기 검증
if file_size > MAX_BLOB_SIZE {
return Err(anyhow!("파일 크기가 최대 허용 크기를 초과합니다"));
```

```
}

// 파일 읽기

std::fs::read(file_path)

}
```

2. RedStuff 인코딩 과정의 수학적 원리

2.1 인코딩 매개변수 계산

Walrus는 비잔틴 장애 허용(Byzantine Fault Tolerance) 환경에서 작동하므로, 인코딩 매개변수는 다음과 같이 계산됩니다:

```
// crates/walrus-core/src/encoding/config.rs

pub fn source_symbols_for_n_shards(n_shards: NonZeroU16) -> (NonZeroU16, NonZe

let f = bft::max_n_faulty(n_shards); // f = ⌊(N-1)/3⌋

let safety_limit = get_safety_limit(n_shards); // σ (안전 한계)

let primary_symbols = n_shards.get() - 2 * f - safety_limit; // N - 2f - σ

let secondary_symbols = n_shards.get() - f - safety_limit; // N - f - σ

(

NonZeroU16::new(primary_symbols).unwrap(),

NonZeroU16::new(secondary_symbols).unwrap(),

)

}
```

수학적 배경:

- **N**: 전체 샤드 수
- **f**: 최대 비잔틴 노드 수 (f < N/3)
- **σ**: 안전 한계 (디코딩 실패 확률을 낮추기 위한 추가 여유분)
- **1차 소스 심볼 수**: N - 2f - σ (가장 제한적인 조건)
- **2차 소스 심볼 수**: N - f - σ (덜 제한적인 조건)

2.2 심볼 크기 계산

```
// crates/walrus-core/src/encoding/utils.rs
pub fn compute_symbol_size(
blob_size: u64,
source_symbols_per_blob: NonZeroU32,
required_alignment: u16,
) -> Result<NonZeroU16, DataTooLargeError> {
// 심볼 크기 = [블롭_크기 / 총_소스_심볼_수]
let symbol_size = (blob_size + u64::from(source_symbols_per_blob.get()) - 1)
/ u64::from(source_symbols_per_blob.get());
// 정렬 요구사항에 맞춰 조정
let aligned_size = ((symbol_size + u64::from(required_alignment) - 1)
/ u64::from(required_alignment)) * u64::from(required_alignment);
NonZeroU16::try_from(aligned_size).map_err(|_| DataTooLargeError)
}
```

2.3 메시지 매트릭스 구성

블롭 데이터는 직사각형 메시지 매트릭스로 배열됩니다:

```
// crates/walrus-core/src/encoding/blob_encoding.rs

impl<'a> BlobEncoder<'a> {

pub  fn  new(config: EncodingConfigEnum<'a>, blob: &'a [u8]) -> Result<Self, Dat

let  symbol_size = utils::compute_symbol_size_from_usize(

blob.len(),

config.source_symbols_per_blob(),

config.encoding_type().required_alignment(),

)?;

let  n_rows = config.n_source_symbols::<Primary>().get().into(); // N - 2f - σ

let  n_columns = config.n_source_symbols::<Secondary>().get().into(); // N - f -

Ok(Self {

blob,

symbol_size,

n_rows,

n_columns,

config,

span: tracing::span!(Level::ERROR, "BlobEncoder", blob_size = blob.len()),

})

}
```

```
}
```

매트릭스 구조:

```
메시지 매트릭스 (n_rows × n_columns):

┌──────────────────────────────────────┐

│ s₀,₀ s₀,₁  s₀,₂  ... s₀,ₙ₋₁  │ ← 행 0 (1차 슬리버 0의 소스)

│ s₁,₀ s₁,₁  s₁,₂  ... s₁,ₙ₋₁  │ ← 행 1 (1차 슬리버 1의 소스)

│ ⋮ ⋮ ⋮ ⋱ ⋮ │

│ sₘ,₀ sₘ,₁  sₘ,₂  ... sₘ,ₙ₋₁  │ ← 행 m (1차 슬리버 m의 소스)

└──────────────────────────────────────┘

↑ ↑ ↑ ↑

열0 열1 열2 열n-1

(2차 슬리버들의 소스)
```

3. 슬리버 생성 과정

3.1 1차 및 2차 인코딩

```rust
impl<'a> BlobEncoder<'a> {

pub fn encode(&self) -> Vec<SliverPair> {

let mut primary_slivers: Vec<_> = self.empty_slivers::<Primary>();

let mut secondary_slivers: Vec<_> = self.empty_slivers::<Secondary>();
```

```
// 1. 시스템적 부분 복사 (원본 데이터)
for (row, sliver) in  self.rows().zip(primary_slivers.iter_mut()) {
sliver.symbols.data_mut()[..row.len()].copy_from_slice(row);
}
for (column, sliver) in  self.column_symbols().zip(secondary_slivers.iter_mut())
sliver.symbols.to_symbols_mut()
.zip(column)
.for_each(|(dest, src)| dest[..src.len()].copy_from_slice(src))
}


// 2. 1차 인코딩: 열(2차 슬리버)을 인코딩하여 1차 복구 심볼 생성
for (col_index, column) in  secondary_slivers.iter().take(self.n_columns).enumer
for (symbol, sliver) in  self
.config
.encode_all_repair_symbols::<Primary>(column.symbols.data())
.expect("크기가 이미 확인됨")
.into_iter()
.zip(primary_slivers.iter_mut().skip(self.n_rows))
{
sliver.copy_symbol_to(col_index, &symbol);
}
}
```

```
// 3. 2차 인코딩: 행(1차 슬리버)을 인코딩하여 2차 복구 심볼 생성
for (r, row) in  primary_slivers.iter().take(self.n_rows).enumerate() {
for (symbol, sliver) in  self
.config
.encode_all_repair_symbols::<Secondary>(row.symbols.data())
.expect("크기가 이미 확인됨")
.into_iter()
.zip(secondary_slivers.iter_mut().skip(self.n_columns))
{
sliver.copy_symbol_to(r, &symbol);
}
}


// 4. 슬리버 쌍 생성
primary_slivers
.into_iter()
.zip(secondary_slivers.into_iter().rev())
.map(|(primary, secondary)| SliverPair { primary, secondary })
.collect()
}
}
```

3.2 RaptorQ 기반 인코딩 구현

```rust
// crates/walrus-core/src/encoding/basic_encoding.rs

impl ReedSolomonEncoder {

pub fn encode_all_repair_symbols(&self) -> Result<Vec<Vec<u8>>, EncodeError> {

let mut repair_symbols = Vec::new();

// RaptorQ 인코더를 사용하여 복구 심볼 생성

for repair_index in self.n_source_symbols.get()..self.n_shards.get() {

let repair_symbol = self.engine

.encode_repair_symbol(repair_index.into(), &self.data)

.map_err(|e| EncodeError::RaptorQError(e.to_string()))?;

repair_symbols.push(repair_symbol);

}

Ok(repair_symbols)

}

}
```

RaptorQ 인코딩의 핵심 특성:

1. **시스템적 코드**: 처음 k개 심볼은 원본 데이터와 동일
2. **선형 코드**: 인코딩된 심볼들은 원본 심볼들의 선형 결합
3. **거의 최적**: k+H개 심볼로 디코딩 실패 확률 < 1/256^(H+1)

3.3 확장된 메시지 매트릭스

```rust
struct ExpandedMessageMatrix<'a> {
```

```
matrix: Vec<Symbols>, // N × N 매트릭스

blob: &'a [u8],

config: &'a EncodingConfigEnum<'a>,

n_rows: usize, // 원본 행 수 (N - 2f - σ)

n_columns: usize, // 원본 열 수 (N - f - σ)

symbol_size: NonZeroU16,

}



impl<'a> ExpandedMessageMatrix<'a> {

fn  new(config: &'a EncodingConfigEnum<'a>, symbol_size: NonZeroU16, blob: &'a [

// 1. N × N 영 매트릭스 생성

let  matrix = vec![

Symbols::zeros(config.n_shards_as_usize(), symbol_size);

config.n_shards_as_usize()

];

let  mut  expanded_matrix = Self { matrix, blob, config, /* ... */ };

// 2. 시스템적 부분 채우기

expanded_matrix.fill_systematic_with_rows();

// 3. 행 확장 (2차 인코딩)

expanded_matrix.expand_rows_for_secondary();

// 4. 열 확장 (1차 인코딩)

expanded_matrix.expand_all_columns();

expanded_matrix
```

```
}

}
```

확장된 매트릭스 구조:

```
완전 확장된 메시지 매트릭스 (N × N):

┌──────────────────────────────────────┬──────────────────┐
│ s₀,₀ s₀,₁  ... s₀,ₙ₋ₓ₋₁  │ r₀,ₙ₋ₓ ... r₀,ₙ₋₁  │ ← 시스템적 행들
│ s₁,₀ s₁,₁  ... s₁,ₙ₋ₓ₋₁  │ r₁,ₙ₋ₓ ... r₁,ₙ₋₁  │
│ ⋮ ⋮ ⋱ ⋮ │ ⋮ ⋱ ⋮ │
│ sₘ,₀ sₘ,₁  ... sₘ,ₙ₋ₓ₋₁  │ rₘ,ₙ₋ₓ ... rₘ,ₙ₋₁  │
├──────────────────────────────────────┼──────────────────┤
│ rₘ₊₁,₀ rₘ₊₁,₁  ... rₘ₊₁,ₙ₋ₓ₋₁  │ 계산된 복구 심볼들 │ ← 복구 행들
│ ⋮ ⋮ ⋱ ⋮ │ │
│ rₙ₋₁,₀ rₙ₋₁,₁  ... rₙ₋₁,ₙ₋ₓ₋₁  │ │
└──────────────────────────────────────┴──────────────────┘

↑ ↑

시스템적 열들 복구 열들
```

4. 메타데이터 생성 및 머클 트리 구성

4.1 슬리버별 머클 트리 생성

각 슬리버에 대해 개별적으로 머클 트리가 구성됩니다:

```rust
// crates/walrus-core/src/encoding/slivers.rs

impl<T: EncodingAxis> SliverData<T> {

pub fn get_merkle_root<U: HashFunction<DIGEST_LEN>>(

&self,

config: &EncodingConfigEnum,

) -> Result<Node, RecoverySymbolError> {

// 1. 슬리버를 N개 심볼로 확장

let recovery_symbols = self.recovery_symbols(config)?;

// 2. 각 심볼에 대해 리프 해시 계산

let leaf_hashes: Vec<_> = recovery_symbols

.to_symbols()

.map(|symbol| leaf_hash::<U>(symbol))

.collect();

// 3. 머클 트리 구성 및 루트 반환

Ok(MerkleTree::<U>::build_from_leaf_hashes(leaf_hashes.into_iter()).root())

}

}
```

4.2 블롭 메타데이터 구성

```rust
impl<'a> ExpandedMessageMatrix<'a> {

fn get_metadata(&self) -> VerifiedBlobMetadataWithId {

let n_shards = self.config.n_shards_as_usize();
```

```
// 1. 모든 심볼에 대한 리프 해시 계산
let mut leaf_hashes = Vec::with_capacity(n_shards * n_shards);
for row in 0..n_shards {
for col in 0..n_shards {
leaf_hashes.push(leaf_hash::<Blake2b256>(&self.matrix[row][col]));
}
}

// 2. 각 슬리버 쌍에 대한 메타데이터 생성
let mut metadata = Vec::with_capacity(n_shards);
for sliver_index in 0..n_shards {
// 1차 슬리버 머클 루트
let primary_hash = MerkleTree::<Blake2b256>::build_from_leaf_hashes(
leaf_hashes[n_shards * sliver_index..n_shards * (sliver_index + 1)]
.iter()
.cloned(),
).root();
// 2차 슬리버 머클 루트 (열 순서 역순)
let secondary_hash = MerkleTree::<Blake2b256>::build_from_leaf_hashes(
(0..n_shards).map(|symbol_index| {
leaf_hashes[n_shards * symbol_index + n_shards - 1 - sliver_index].clone()
}),
).root();
```

```
metadata.push(SliverPairMetadata {
primary_hash,
secondary_hash,
});
}

// 3. 블롭 메타데이터 생성
VerifiedBlobMetadataWithId::new_verified_from_metadata(
metadata,
self.config.encoding_type(),
u64::try_from(self.blob.len()).expect("유효한 블롭 크기"),
)
}
}
```

4.3 블롭 ID 계산

```
// crates/walrus-core/src/metadata.rs
impl BlobId {
pub fn from_sliver_pair_metadata(metadata: &BlobMetadata) -> Self {
// 1. 슬리버 쌍 메타데이터들을 연결
let concatenated_hashes: Vec<_> = metadata
.hashes()
```

```
.iter()
.map(|pair_meta| pair_meta.pair_leaf_input::<Blake2b256>())
.collect();
// 2. 연결된 해시들로 머클 트리 구성
let sliver_pairs_merkle_root = MerkleTree::<Blake2b256>::build(
concatenated_hashes.iter().map(|input| input.as_slice())
).root();
// 3. 최종 블롭 ID 계산
let mut hasher = Blake2b256::new();
hasher.update(&[metadata.encoding_type() as u8]); // 인코딩 타입
hasher.update(&metadata.unencoded_length().to_le_bytes()); // 원본 크기
hasher.update(sliver_pairs_merkle_root.as_bytes()); // 머클 루트
Self(hasher.finalize().into())
}
}
```

5. 병렬 처리 및 최적화 기법

5.1 병렬 인코딩

```
// crates/walrus-sdk/src/client.rs
impl WalrusNodeClient {
pub fn encode_blobs<'a, T: Debug + Clone + Send + Sync>(
```

```
&self,
walrus_store_blobs: Vec<WalrusStoreBlob<'a, T>>,
encoding_type: EncodingType,
) -> ClientResult<Vec<WalrusStoreBlob<'a, T>>> {
let multi_pb = Arc::new(MultiProgress::new());


// 병렬 인코딩 실행
let results = walrus_store_blobs
.into_par_iter() // Rayon을 사용한 병렬 처리
.map(|blob| {
let multi_pb_clone = multi_pb.clone();
let unencoded_blob = blob.get_blob();
let encode_result = self.encode_pairs_and_metadata(
unencoded_blob,
encoding_type,
multi_pb_clone.as_ref(),
);
blob.with_encode_result(encode_result)
})
.collect::<Vec<_>>();


// 결과 수집 및 오류 처리
let mut final_results = Vec::with_capacity(results.len());
```

```
for result in results {
final_results.push(result?);
}

Ok(final_results)
}
}
```

5.2 SIMD 최적화

RaptorQ 구현에서는 SIMD 명령어를 활용한 최적화가 적용됩니다:

```
// reed-solomon-simd 크레이트 내부 (개념적 구현)
impl ReedSolomonEngine {
#[cfg(target_arch = "x86_64")]
fn encode_symbol_avx2(&self, data: &[u8], coefficients: &[u8]) -> Vec<u8> {
unsafe {
// AVX2 명령어를 사용한 벡터화된 갈루아 필드 연산
let mut result = vec![0u8; data.len()];
for i in (0..data.len()).step_by(32) {
let data_vec = _mm256_loadu_si256(data.as_ptr().add(i) as *const __m256i);
let coeff_vec = _mm256_set1_epi8(coefficients[i / 32] as i8);
let product = gf_mul_avx2(data_vec, coeff_vec);
_mm256_storeu_si256(result.as_mut_ptr().add(i) as *mut __m256i, product);
```

```
}
result
}
}
}
```

5.3 메모리 최적화

```
impl<'a> ExpandedMessageMatrix<'a> {
/// 복구 심볼 부분을 메모리에서 해제하여 메모리 사용량 최적화
fn drop_recovery_symbols(&mut self) {
self.matrix
.iter_mut()
.skip(self.n_rows)
.for_each(|row| row.truncate(self.n_columns));
}
/// 슬리버 쌍 작성 시 매트릭스 소비하여 메모리 복사 최소화
fn write_primary_slivers(self, sliver_pairs: &mut [SliverPair]) {
for (sliver_pair, mut row) in sliver_pairs.iter_mut().zip(self.matrix.into_ite
row.truncate(self.config.n_source_symbols::<Secondary>().get().into());
sliver_pair.primary.symbols = row; // 이동 시맨틱 활용
}
}
```

```
}
```

6. 슬리버 배포 과정

6.1 스토리지 노드 선택

```
// crates/walrus-sdk/src/client.rs

impl WalrusNodeClient {

async fn reserve_and_store_encoded_blobs<'a, T: Debug + Clone + Send + Sync +

&'a self,

encoded_blobs: Vec<WalrusStoreBlob<'a, T>>,

store_args: &StoreArgs,

) -> ClientResult<Vec<WalrusStoreBlob<'a, T>>> {

// 1. 현재 위원회 정보 가져오기

let committees = self.get_committees().await?;

// 2. 블롭 상태 확인

let encoded_blobs_with_status = self

.await_while_checking_notification(self.get_blob_statuses(encoded_blobs))

.await?;

// 3. 블록체인에 블롭 등록

let registered_blobs = self

.resource_manager(&committees)

.await
```

```
.register_walrus_store_blobs(
encoded_blobs_with_status,
store_args.epochs_ahead,
store_args.persistence,
store_args.store_optimizations,
)
.await?;
// 4. 슬리버 배포 실행
let stored_blobs = self
.store_blobs_to_storage_nodes(registered_blobs, store_args)
.await?;
Ok(stored_blobs)
}
}
```

6.2 병렬 슬리버 전송

```
impl WalrusNodeClient {
async fn store_blobs_to_storage_nodes<'a, T: Debug + Clone + Send + Sync + 'a>
&'a self,
registered_blobs: Vec<WalrusStoreBlob<'a, T>>,
store_args: &StoreArgs,
) -> ClientResult<Vec<WalrusStoreBlob<'a, T>>> {
```

```
let mut tasks = Vec::new();
for blob in registered_blobs {
if let WalrusStoreBlob::Registered(registered_blob) = &blob {
// 각 슬리버 쌍에 대해 병렬 저장 작업 생성
for (shard_index, sliver_pair) in registered_blob.sliver_pairs.iter().enumerate
let storage_node_url = self.get_storage_node_url(shard_index).await?;
// 1차 슬리버 저장 작업
let primary_task = self.store_sliver_async(
storage_node_url.clone(),
registered_blob.metadata.blob_id(),
SliverPairIndex(shard_index as u16),
&sliver_pair.primary,
);
// 2차 슬리버 저장 작업
let secondary_task = self.store_sliver_async(
storage_node_url,
registered_blob.metadata.blob_id(),
SliverPairIndex(shard_index as u16),
&sliver_pair.secondary,
);
tasks.push(primary_task);
tasks.push(secondary_task);
}
```

```
}
}
// 모든 저장 작업을 병렬로 실행
let results = futures::future::join_all(tasks).await;
// 결과 처리 및 오류 확인
for result in results {
result?;
}
Ok(registered_blobs)
}
}
```

6.3 슬리버 검증 및 저장

스토리지 노드에서의 슬리버 처리:

```
// crates/walrus-service/src/node.rs
impl StorageNode {
async fn store_sliver(
&self,
blob_id: &BlobId,
sliver_pair_index: SliverPairIndex,
sliver: Sliver,
) -> Result<bool, StoreSliverError> {
```

```
// 1. 슬리버 검증
self.verify_sliver(blob_id, &sliver).await?;
// 2. 중복 저장 확인
if self.storage.is_sliver_stored(blob_id, sliver.r#type()).await? {
return Ok(false); // 이미 저장됨
}
// 3. 데이터베이스에 저장
self.storage.store_sliver_data(blob_id, sliver_pair_index, &sliver).await?;
// 4. 메트릭 업데이트
self.metrics.increment_stored_slivers(sliver.r#type());
Ok(true) // 새로 저장됨
}
async fn verify_sliver(&self, blob_id: &BlobId, sliver: &Sliver) -> Result<(),
// 1. 블롭 메타데이터 가져오기
let metadata = self.storage.get_blob_metadata(blob_id).await?;
// 2. 슬리버 크기 검증
let expected_size = self.encoding_config
.sliver_size_for_blob::<Primary>(metadata.unencoded_length())?;
if sliver.data.len() != expected_size.get() as usize {
return Err(VerificationError::SizeMismatch);
}
// 3. 머클 증명 검증
let sliver_data = SliverData::new(
```

```
sliver.data.clone(),
metadata.symbol_size(&self.encoding_config)?,
sliver.index,
);
sliver_data.verify(&self.encoding_config, &metadata)?;
Ok(())
}
}
```

7. 성능 최적화 및 모니터링

7.1 배치 처리 최적화

```
impl StorageNodeClient {
async fn batch_store_slivers(
&self,
requests: Vec<StoreSliverRequest>,
) -> Result<Vec<StoreSliverResponse>, NetworkError> {
// 요청을 노드별로 그룹화
let mut requests_by_node: HashMap<NodeId, Vec<StoreSliverRequest>> = HashMap::
for request in requests {
requests_by_node
.entry(request.node_id)
```

```
.or_default()
.push(request);
}
// 각 노드에 대해 병렬 배치 요청
let batch_futures: Vec<_> = requests_by_node
.into_iter()
.map(|(node_id, node_requests)| {
self.send_batch_request(node_id, node_requests)
})
.collect();
let batch_results = futures::future::join_all(batch_futures).await;
// 결과 병합
let mut all_responses = Vec::new();
for result in batch_results {
all_responses.extend(result?);
}
Ok(all_responses)
}
}
```

7.2 연결 풀링

```
pub struct ConnectionPool {
```

```
pools: HashMap<NodeId, Pool<HttpConnection>>,
max_connections_per_node: usize,
connection_timeout: Duration,
}


impl ConnectionPool {
pub  async  fn  get_connection(&self, node_id: NodeId) -> Result<PooledConnectio
let  pool = self.pools
.get(&node_id)
.ok_or(NetworkError::NodeNotFound)?;
// 타임아웃과 함께 연결 획득
timeout(self.connection_timeout, pool.get())
.await
.map_err(|_| NetworkError::ConnectionTimeout)?
.map_err(NetworkError::from)
}
pub  async  fn  execute_request<T>(
&self,
node_id: NodeId,
request: HttpRequest,
) -> Result<T, NetworkError>
where
T: DeserializeOwned,
```

```
{
let mut connection = self.get_connection(node_id).await?;
let response = connection.send_request(request).await?;
if response.status().is_success() {
Ok(response.json().await?)
} else {
Err(NetworkError::HttpError(response.status()))
}
}
}
```

7.3 진행률 모니터링

```
use indicatif::{MultiProgress, ProgressBar, ProgressStyle};

impl WalrusNodeClient {
pub fn encode_pairs_and_metadata(
&self,
blob: &[u8],
encoding_type: EncodingType,
multi_pb: &MultiProgress,
) -> ClientResult<(Vec<SliverPair>, VerifiedBlobMetadataWithId)> {
```

```
// 진행률 표시기 생성
let spinner = multi_pb.add(ProgressBar::new_spinner());
spinner.set_style(
ProgressStyle::default_spinner()
.template("{spinner:.green} {msg}")
.unwrap()
);
spinner.set_message("블롭 인코딩 중...");
let encode_start = Instant::now();
// 실제 인코딩 수행
let (pairs, metadata) = self
.encoding_config
.get_for_type(encoding_type)
.encode_with_metadata(blob)
.map_err(ClientError::other)?;
let duration = encode_start.elapsed();
// 완료 메시지 표시
spinner.finish_with_message(format!(
"인코딩 완료 ({}ms) - 블롭 ID: {}",
duration.as_millis(),
metadata.blob_id()
));
Ok((pairs, metadata))
```

```
}

}
```

8. 오류 처리 및 복구 메커니즘

8.1 인코딩 오류 처리

```
#[derive(Debug, thiserror::Error)]

pub enum EncodingError {

#[error("블롭 크기가 너무 큼: {size}, 최대: {max}")]

BlobTooLarge { size: usize, max: usize },

#[error("심볼 크기가 유효하지 않음: {size}")]

InvalidSymbolSize { size: u16 },

#[error("RaptorQ 인코딩 실패: {reason}")]

RaptorQFailure { reason: String },

#[error("메모리 부족")]

OutOfMemory,

}


impl BlobEncoder<'_> {

pub fn encode(&self) -> Result<Vec<SliverPair>, EncodingError> {

// 메모리 사용량 사전 확인

let estimated_memory = self.estimate_memory_usage();
```

```
if estimated_memory > self.config.max_memory_usage() {
return Err(EncodingError::OutOfMemory);
}
// 인코딩 실행 (이전 구현과 동일)
// ...
}
fn estimate_memory_usage(&self) -> usize {
let n_shards = self.config.n_shards_as_usize();
let symbol_size = usize::from(self.symbol_size.get());
// 확장된 매트릭스 크기 + 슬리버 쌍들 크기
n_shards * n_shards * symbol_size +
n_shards * (self.n_rows + self.n_columns) * symbol_size
}
}
```

8.2 네트워크 오류 및 재시도

```
impl WalrusNodeClient {
async fn store_sliver_with_retry(
&self,
node_url: Url,
blob_id: &BlobId,
sliver_pair_index: SliverPairIndex,
```

```
sliver: &SliverData<impl EncodingAxis>,
max_retries: usize,
) -> Result<(), StorageError> {
let mut last_error = None;
for attempt in 0..=max_retries {
match self.store_sliver_once(node_url.clone(), blob_id, sliver_pair_index, sliv
Ok(()) => return Ok(()),
Err(e) if e.is_retryable() && attempt < max_retries => {
last_error = Some(e);
// 지수 백오프
let delay = Duration::from_millis(100 * 2_u64.pow(attempt as u32));
tokio::time::sleep(delay).await;
tracing::warn!(
attempt = attempt + 1,
max_retries,
error = %last_error.as_ref().unwrap(),
"슬리버 저장 재시도"
);
}
Err(e) => return Err(e),
}
}
Err(last_error.unwrap_or_else(|| {
```

```
StorageError::MaxRetriesExceeded { max_retries }
}))
}
}
```

9. 성능 메트릭 및 관찰성

9.1 인코딩 성능 메트릭

```
use prometheus::{Counter, Histogram, register_counter, register_histogram};

lazy_static! {
static ref ENCODING_DURATION: Histogram = register_histogram!(
"walrus_encoding_duration_seconds",
"블롭 인코딩 소요 시간",
vec![0.001, 0.01, 0.1, 1.0, 10.0, 60.0]
).unwrap();
static ref ENCODING_THROUGHPUT: Histogram = register_histogram!(
"walrus_encoding_throughput_bytes_per_second",
"인코딩 처리량 (바이트/초)",
prometheus::exponential_buckets(1024.0, 2.0, 20).unwrap()
).unwrap();
static ref SLIVER_STORE_COUNTER: Counter = register_counter!(
```

```
"walrus_slivers_stored_total",
"저장된 슬리버 총 개수"
).unwrap();
}


impl BlobEncoder<'_> {
pub fn encode_with_metrics(&self) -> Vec<SliverPair> {
let start_time = Instant::now();
let blob_size = self.blob.len();
let result = self.encode();
let duration = start_time.elapsed();
let throughput = blob_size as f64 / duration.as_secs_f64();
ENCODING_DURATION.observe(duration.as_secs_f64());
ENCODING_THROUGHPUT.observe(throughput);
tracing::info!(
blob_size,
duration_ms = duration.as_millis(),
throughput_mbps = throughput / 1_048_576.0,
"블롭 인코딩 완료"
);
result
}
```

```
}
```

9.2 분산 추적

```
use tracing::{info_span, Instrument};
use opentelemetry::trace::{TraceContextExt, Tracer};


impl WalrusNodeClient {
#[tracing::instrument(skip(self, blob), fields(blob_id, blob_size = blob.len()))
pub  async  fn  store_blob_with_tracing(
&self,
blob: &[u8],
store_args: &StoreArgs,
) -> ClientResult<BlobStoreResult> {
let  span = info_span!("store_blob", blob_size = blob.len());
async  move {
// 1. 인코딩 단계
let (pairs, metadata) = self
.encode_pairs_and_metadata(blob, store_args.encoding_type, &MultiProgress::new()
.instrument(info_span!("encode_blob"))
.await?;
tracing::Span::current().record("blob_id", &tracing::field::display(metadata.blo
// 2. 블록체인 등록 단계
```

```
let registration_result = self
.register_blob_on_chain(&metadata, store_args)
.instrument(info_span!("register_on_chain"))
.await?;
// 3. 슬리버 배포 단계
let store_result = self
.distribute_slivers(&pairs, &metadata)
.instrument(info_span!("distribute_slivers"))
.await?;
Ok(BlobStoreResult::new(metadata.blob_id(), store_result))
}
.instrument(span)
.await
}
}
```

결론

Walrus의 데이터 저장 프로세스는 수학적으로 엄밀한 RedStuff 인코딩 알고리즘을 기반으로 하여, 높은 가용성과 내구성을 보장하면서도 효율적인 스토리지 오버헤드를 달성합니다.

핵심 특징:

- **비잔틴 장애 허용**: f < N/3 조건에서 안전한 데이터 저장
- **효율적인 복구**: 노드 장애 시 O(S/N) 통신으로 복구 가능
- **병렬 처리**: 인코딩과 배포 과정의 최적화된 병렬화

- **강력한 검증**: 머클 트리 기반 데이터 무결성 보장

이러한 설계를 통해 Walrus는 탈중앙화 환경에서도 기업급 스토리지 시스템의 신뢰성과 성능을 제공할 수 있습니다.